GitHub: `https://github.com/1amKhush/freshroute-rl`

Team: Khushvendra Singh (B24CM1075) Aditya Ranjan (B24CM1004)

FreshRoute RL

Staleness-Aware Read Routing for Distributed Databases

AI Assignment 2: Reinforcement Learning Mini Project

Abstract. FreshRoute RL learns which database replica should answer each read across different geographic regions. A simulator with fixed random seeds models how routing affects queues and asynchronous replication over time. A Deep Q-Network (DQN) balances response time, data freshness, and read failures. Tests on unseen random seeds compare learned routing with fixed rules. An interactive laboratory shows the observations, action values, and outcomes behind each decision.

1. Practical problem and objective

Reading from the nearest replica can return stale data. Sending every read to the write leader can overload its queue. The system has replicas in Mumbai, Frankfurt (the leader), and Virginia. Each request specifies a client region, an allowed timestamp lag b_t, and a response deadline d_t. The goal is to learn a routing policy that maximizes expected discounted reward,

$$J(\pi) = \mathbb{E}_\pi\left[\sum_{t\geq 0}\gamma^t r_t\right], \qquad \gamma = 0.95.$$

Each choice changes later queue lengths and how followers apply writes, making this a sequential control problem. Both agents learn by interacting with the environment. Fixed routing rules are used only for comparison.

2. Reinforcement learning formulation

State space $\mathcal{S}$. The agent receives a bounded observation $s_t \in \mathcal{S} \subseteq [0,1]^{26}$: three client-region and three request-class one-hot entries, $b_t/500$, $d_t/240$, and six features for each of three replicas. For replica i, these are

$$\begin{aligned}\phi_{i,t} = \big(&u_{i,t},\ \mathrm{RTT}_{i,t}/250,\ q_{i,t}/(q_{i,t}+100),\\ &\tau_{i,t}/(\tau_{i,t}+200),\ v_{i,t}/(v_{i,t}+10),\\ &\delta_{i,t}/(\delta_{i,t}+200)\big),\end{aligned}$$

where u is online status, q is queued work, τ is timestamp lag, v counts committed versions not yet applied, and δ is replication delay. Times are in milliseconds. Thus $3+3+2+3(6) = 26$. Freshness is measured as $\tau_{i,t} = T_{\mathrm{latest},t} - T_{\mathrm{applied}(i),t}$, using commit timestamps. A replica that has applied all writes has zero lag, even if no write is recent. Write histories and workload phase are hidden, so this summary is not guaranteed to be a complete Markov state.

Action space $\mathcal{A}$. The discrete space is $\{0,1,2\}$: read from Mumbai, Frankfurt, or Virginia, respectively. An offline replica can still be chosen, so the agent must learn the resulting failure cost. Retries and quorum reads are not available.

Reward function R. For $i = a_t$, latency is computed as $\widehat{\ell}_t = \mathrm{RTT}_{i,t} + q_{i,t} + w_i$, where $(w_0, w_1, w_2) = (14, 22, 14)$ ms. The failure flag and recorded latency are

$$f_t = \mathbf{1}\{u_{i,t} = 0 \text{ or } \widehat{\ell}_t > d_t\}, \qquad \ell_t = (1-f_t)\widehat{\ell}_t + f_t d_t.$$

With $[x]_+ = \max(x, 0)$, the implemented reward is

$$\boxed{r_t = -\frac{\ell_t}{d_t} - 2(1-f_t)\min\left(\frac{[\tau_{i,t}-b_t]_+}{\max(b_t,50)}, 2\right) - 4f_t.}$$

Dividing latency by the deadline makes costs comparable across requests. Staleness beyond the budget gets a capped penalty; the 50 ms floor handles zero-budget requests. Failures receive reward -5 and no freshness credit. Joint service-level agreement (SLA) success is $\mathbf{1}\{f_t = 0, \tau_{i,t} \leq b_t\}$: the reward encourages freshness but does not enforce a hard consistency constraint.

3. Environment and learning algorithm

The custom Gymnasium simulator advances by 12 ms per request. Each read adds service work to the chosen online replica, even if the read times out. Online replicas drain their queues and receive background work; follower replication delay also depends on queue pressure. Writes, asynchronous version replay, failures, and workload changes produce the next observation. Episodes truncate after 160 reads; learning still bootstraps future value at this time limit. Reads use snapshots taken at dispatch, with these request classes:

Class	Lag budget b_t (ms)	Deadline d_t (ms)
Latest	0	240
Bounded	100	180
Relaxed	500	140

The main agent is vanilla DQN [1, 2]: a $26 \to 128 \to 128 \to 3$ network with experience replay and a target network. It learns by minimizing Huber error against the one-step target

$$y_t = r_t + \gamma(1-z_t)\max_{a'} Q_{\theta^-}(s_{t+1}, a'),$$

where z_t marks true termination, not the episode time limit. Learning rate is 10^{-4}, replay capacity 100,000, batch size 64, target update interval 1,000 steps, and one gradient update follows every four interactions after a 5,000-step warm-up. Exploration decreases from $\epsilon = 1$ to 0.05 over the first 70% of training; evaluation uses $\arg\max_a Q_\theta(s,a)$. DQN handles real-valued telemetry without listing every possible state combination.

A separate tabular Q-learning agent [3] groups region, request class, availability, queue, and freshness into coarse buckets, with $\alpha = 0.12$, $\gamma = 0.95$, and 192,000 demonstration interactions. It is not included as a paired baseline in the final DQN tests.

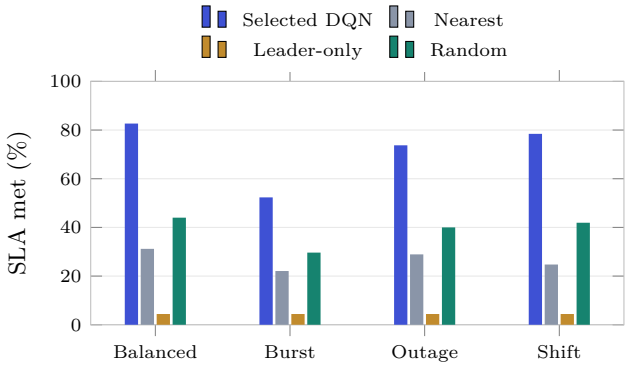


Figure 1: Deadline-and-freshness success on unseen test seeds for the validation-selected DQN and fixed rules: 30 episodes per workload.

4. Experimental protocol and results

Training includes balanced traffic, write bursts, regional outages, and shifts in client traffic. Five full-observation DQNs use seeds 7, 23, 47, 71, and 97; each ablation uses 7, 23, and 47. Every model receives 50,000 interactions: 14 models and 700,000 interactions in total. Validation workloads select checkpoints. A separate 30-seed validation matrix selects demonstration seed 71 using mean reward with equal weight for all four workloads. The selection and model hashes are fixed before testing.

Final testing [4] uses seeds 200000–200029 in each regime, with 160 requests per episode. Policies receive the same external workload draws, while their own queues evolve separately. Controls are *nearest*, *leader-only*, and uniform *random*; none provides training labels or a hidden routing fallback.

The selected DQN meets both freshness and deadline requirements on 82.7% of balanced requests, versus 31.2% for nearest and 44.0% for random (Fig. 1); its mean reward is -0.763 and response availability is 98.8%. Leader-only meets the SLA on 4.4% of requests because sending all reads to the leader overloads its queue at these service rates. Availability means a data response within the deadline, not replica uptime.

Across all five full-agent seeds, macro SLA satisfaction is 71.0% and mean reward is -1.281. Figure 2 compares separately trained ablations: no-freshness sets τ, v, δ channels to zero; no-queue zeros q, δ; $\gamma = 0$ keeps all observation features. Queue and replication information help in this setup. The small freshness-ablation gap does not show that these features are always needed. Discount-ablation differences can also come from optimization effects. Intervals resample mean results across training seeds, rather than treating correlated requests as independent.

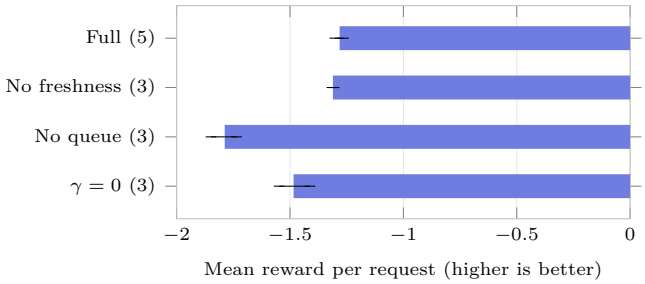


Figure 2: Mean test reward with equal weight for all four workloads. Parentheses count independent training seeds; whiskers show descriptive 95% percentile bootstrap intervals across those agents.

5. Interactive system and reproducibility

A React interface and FastAPI backend show actual observations, the three learned action values, chosen routes, and measured reward components. Finite packet animations show fresh, stale, and failed reads, with pause, stepping, and reduced-motion controls. Live sessions run a frozen policy and let users inject outages, lag, and load. Eight-step counterfactuals copy the same simulator and random state to try different first actions without changing the original session. Q-values are estimates; branch outcomes are measured. The repository includes runnable code, dependency locks, trained agents, reproduction scripts, and checksummed traces.

6. Scope and limitations

These results come from a synthetic simulator using new seeds within the same four workloads. They do not establish convergence or production performance. DQN and tabular Q-learning were not compared on paired final tests. Telemetry is exact; noisy monitoring, mid-flight failures, leader election, hard consistency enforcement, and real-database validation remain future work. Zero timestamp lag alone does not establish linearizability.

References

[1] V. Mnih et al., "Playing Atari with Deep Reinforcement Learning," arXiv:1312.5602, 2013. `https://arxiv.org/abs/1312.5602`

[2] Stable-Baselines3, "DQN," version 2.9.0 documentation. `https://stable-baselines3.readthedocs.io/en/v2.9.0/modules/dqn.html`

[3] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, pp. 279–292, 1992. `https://www.gatsby.ucl.ac.uk/~dayan/papers/cjch.pdf`

[4] K. Singh and A. Ranjan, *FreshRoute RL*, source code and frozen experiment artifacts, 2026. See `artifacts/releases/evaluation.json`, `test-matrix.json.gz`, and `frozen.json`. `https://github.com/1amKhush/freshroute-rl`